FDEVS 2025 – Trilha Backend

Ana Julia Reis de Mattos

Sumário

1. GIT E GITHUB 2
1.1. CONCEITOS GIT 2
1.2. COMANDOS GIT 4
1.3. BOAS PRÁTICAS DE COMMITS 15
1.4. ESTRUTURA DE COMMIT 16
1.5. BRANCHES 17
1.6. PROCESSOS GIT 19
1.6.1. SUBIR PROJETO DO COMPUTADOR PARA O GITHUB 19
1.6.2. BAIXAR PROJETO DO GITHUB PARA O COMPUTADOR 19
1.6.3. COMPARANDO REPOSITÓRIOS LOCAL E REMOTO 20
1.6.4. CONFLITOS 21
1.6.5. README.MD 23
1.7. NA PRÁTICA ..... **Error! Bookmark not defined.**
REFERÊNCIAS 24

1. GIT E GITHUB

1.1. CONCEITOS GIT

- **GIT -** É um Sistema de Controle de Versão (VCS - Version Control System) utilizado para registrar, acompanhar e gerenciar o histórico de alterações em arquivos de um projeto. É essencial para o trabalho em equipe, permitindo que várias pessoas colaborem em um mesmo projeto de forma segura e organizada;

- **GITHUB** - É uma plataforma de hospedagem de código em nuvem, baseada no Git. Ela permite armazenar repositórios online, compartilhar projetos, colaborar com outros desenvolvedores e revisar código de forma prática;

- **REPOSITÓRIO LOCAL -** É a pasta no computador do desenvolvedor, manipulada com Git, onde ficam os arquivos do projeto e o histórico de versões;

- **REPOSITÓRIO REMOTO -** É a versão online de um repositório, hospedada em servidores como o GitHub, GitLab ou Bitbucket. Ele permite o compartilhamento e sincronização do projeto com outros colaboradores**;**

- **DIRETÓRIO DE TRABALHO CORRENTE (WORKING DIRECTORY):** É o diretório local onde são feitas as alterações por prompts;

- **ALTERAÇÕES ELENCADAS PARA O COMMIT (STAGING SNAPSHOT OU STAGE):** É o local onde você separa e escolhe quais alterações vão ser incluídas no próximo commit;

- **COMMITS FEITOS (COMMIT HISTORY) -** É o histórico de commits em ordem cronológica. É possível visualizar esse histórico através do comando GIT LOG;

- **PULL REQUESTS(PRs) -** São um pedido para que o seu código seja incluído num projeto. O processo envolve alterar o código na sua *branch*, fazer um *push* para a sua *branch* no GitHub e, em seguida, pedir o Pull Request da sua *branch* no GitHub para a *branch* original (de desenvolvimento ou de longo prazo);

- **Code Review (Revisão de Código)** - Processo em que outros programadores revesam o seu código para verificar se ele segue as boas práticas de desenvolvimento, convenções internas da equipe e para identificar possíveis erros ou melhorias

1.2. COMANDOS GIT

- **GIT CONFIG:** Configura informações do usuário:
  - ***Para todos os repositórios:***
    git config --global user.name "Seu Nome"
    git config --global user.email "Seu Nome"
  - ***Para o repositório atual apenas:***
    git config user.name "Seu Nome"
    git config user.email "Seu Nome"
  - ***Para conferir configurações:***
    git config --local –list

- **GIT INIT:** Cria um repositório Git vazio na pasta atual;

- **GIT BRANCH:** Gerencia as ramificações no repositório:
  - ***Ver Branchs:*** *git branch*
  - ***Criar Branch:*** *git branch nome-da-branch*

- **GIT CHECKOUT:** Muda de Branch ou restaura arquivos:
  - ***Trocar Branch:*** ***git checkout nome-branch***
  - ***Criar e acessar Branch:*** *git checkout -b nome-branch*

- **GIT STATUS:** Mostra o estado atual do repositório;

- **GIT CLONE:** Copia um repositório remoto para seu computador:
  - *Git clone link-repositorio*

- **GIT ADD:** Adiciona arquivos a serem commitados:
  - ***Adiciona todos os arquivos:*** *git add .*
  - ***Adiciona arquivos específicos:*** *git add arquivo.extensao*

- **GIT COMMIT:** Cria um novo Commit:
  git commit -m "Mensagem descritiva"

- **git commit -m + git add.**
  git commit -am "Mensagem descritiva"

- **GIT PUSH:** Envia seus commits para o repositório remoto:
  - *git push origin nome-branch*

- **GIT PULL:** Baixa e incorpora (faz merge) das mudanças do repositório remoto:
  - *git pull origin nome-branch*

- **GIT MERGE:** Une duas branches;

- **GIT DIFF:** Mostra a diferença entre arquivos modificados e o último commit;
  - **Visualizar o que tem na** *nome_branch_a_comparar mas não tem na nome_branch_base*

  git diff *nome_branch_base nome_branch_a_comparar*

  - **Visualizar o que tem na** *nome_branch_base mas não tem na nome_branch_a_comparar*

  git diff *nome_branch_a_comparar nome_branch_base*

  - **(..) - Visualizar diferenças entre os últimos commits de cada branch** *(locais)*

  git diff *nome_branch_base..nome_branch_a_comparar*

  - **(...) - Visualizar diferenças desde o ponto em que as branchs** *(locais)* **começam a divergirem**

  git diff *nome_branch_base...nome_branch_a_comparar*

  - **(...) - Visualizar diferenças entre branch local que não foram enviadas para o repositório remoto**

git diff origin/*nome_branch_remota nome_branch_local*

- **GIT LOG:** Mostra o histórico de Commits:

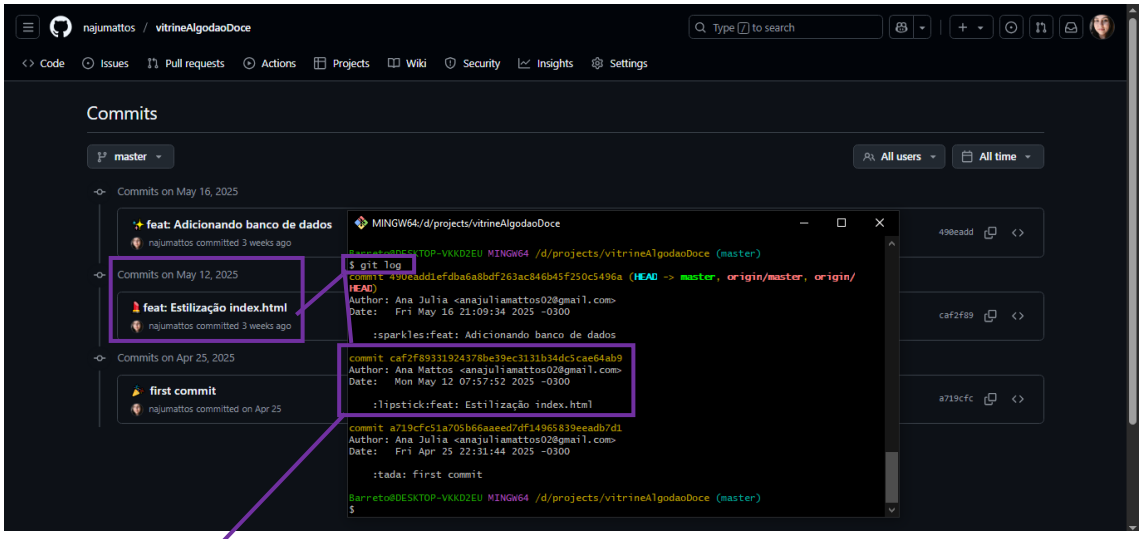

Figura 1 - https://github.com/najumattos/vitrineAlgodaoDoce/commits/master/

- **Abv. Hash (ID único) do Commit:** *a719cfc*
- **Autor do Commit:** *Ana Julia*
- **Data e Hora:** *25/04/2025 22:31:44*
- **Mensagem do Commit:** *first commit*

- **git log --pretty=oneline** : Exibe apenas a abreviação do Hash e mensagem do commit;
- **git log –stat** : Exibe todas as informações e alterações que foram feitas por esse commit de forma abreviada;
- **git log –patch** : Exibe a diferença entre os arquivos alterados pelo commit;
- **git log –graph** : Exibe o histórico de branches e merges no gráfico ASCII;
- **git log -n *número-limite*** **:** Limita o número de commits exibidos;
- **git log --author="*Nome-do-Autor*"** **:** Filtra os commits pelo nome do autor;
- **git log --grep="*palavra-chave*"**: Filtra os commits pela mensagem do commit;

- **git log --follow** *nome-do-arquivo* **:** Mostra o histórico de commits de um arquivo específico;
- **git log** *nome_branch_base..nome_branch_a_comparar* : Lista commits da branch_a_comparar que não estão na branch_base;
- **git log --oneline --graph --decorate** *nome_branch_base nome_branch_a_comparar* : Visualizar o histórico de commits de ambas as branches *(locais)* e onde elas se divergem e se unem.
- **git log** *nome_da_branch_local***..origin/***nome_da_branch_remota* : Mostra os commits que estão na ***remota*** mas não na ***local***.
- **git log origin/***nome_da_branch_remota***..***nome_da_branch_local* : Mostra os commits que estão ***local*** mas não na ***remota.***

- **GIT STASH:** Salva temporariamente alterações de forma paralela ao commit; utilizado para gerenciar mudanças não comitadas de forma flexível, sem perder o seu progresso.

  Quando Usar?

  - Para mudar para outra branch (por exemplo, para corrigir um bug de alta prioridade ou para revisar o código de um colega), o stash permite fazer isso sem comprometer o trabalho atual;
  - Para atualizar a branch local com alterações do repositório remoto (git pull), mas tem modificações locais que podem causar conflito utiliza-se o stash primeiro;
  - Para limpar o diretório de trabalho para rodar testes, depurar algo, ou experimentar uma ideia sem perder as modificações atuais;
  - Para guardar códigos que não serão utilizados no momento mas talvez sejam úteis depois sem poluir o histórico do Git.

- **GIT STASH POP:** Restaura as alterações que foram previamente salvas, aplicando-as de volta ao diretório de trabalho e removendo-as da lista de *stashes*.
- **GIT REVERT:** Cria um novo commit que desfaz as alterações de um *commit* anterior específico adicionando um novo registro ao histórico,

revertendo as modificações de um *commit* indesejado sem apagar ou reescrever o histórico de commits:

git revert *hash_do_commit*

Quando Utilizar?

- Para manter um registro claro de todas as ações, inclusive as de desfazer permitindo a rastreabilidade e evitando a reescrita de um histórico que outros já baixaram**;**
- Para evita**r** quebrar o histórico de colaboradores caso um *commit* contendo um *bug* ou uma funcionalidade indesejada já foi enviado para um repositório remoto (e outros membros da equipe já sincronizaram esse *commit*);
- Para desfazer um erro introduzido em um *commit* específico sem afetar *commits* posteriores que talvez sejam válidos;
- Para facilitar a auditoria e compreensão das mudanças ao longo do tempo já que Cada reversão é um novo *commit*, o que adiciona uma camada de clareza ao que foi desfeito e por que.

Se surgir conflitos CALMA!!!!!!!!

- **Se as alterações do *commit* que está sendo revertido entram em conflito com o estado atual da *branch*, o Git pausará o processo de reversão.**
  - **Edite os arquivos afetados;**
  - **Adicione os arquivos ao *staging area* (git add <arquivo>);**
  - **continue a reversão com git revert –continue.**

- **GIT RESET** - Permite desfazer *commits* alterando assim o histórico da *branch* atual. O git revert cria um novo *commit e o* git reset remove os *commits* desfeitos do histórico da *branch*.

  Quando Utilizar?

- Para desfazer erros *Commits* Recentes que foram feitos localmente e ainda não foram enviados para um repositório remoto. Isso corrige erros sem afetar o histórico público;
- Para combinar vários *commits* em um só, ou dividir um *commit* grande em vários menores, antes de publicá-los;
- Para remover Arquivos caso um arquivo muito grande foi comitado por engano.
- Para descarta todas as modificações locais e retornar o diretório de trabalho a um ponto específico e limpo do histórico.

git reset --soft *hash_do_commit_alvo* **-** Move o HEAD para o *commit* alvo mantendo as alterações dos *commits* "desfeitos" no *staging area* (prontas para um novo *commit*) de modo que o diretório de trabalho permanece inalterado permitindo reagrupar *commits* ou reescrever a mensagem de um *commit* sem perder as alterações de código;

git reset --mixed *hash_do_commit_alvo* **-** Move o HEAD para o *commit* alvo e limpa o *staging area*. As alterações dos *commits* "desfeitos" retornam para o diretório de trabalho como modificações não *stage* sendo o modo padrão já que permite refazer o *commit* a partir do zero e selecionar quais arquivos serão adicionados ao *staging area* novamente;

git reset --hard *hash_do_commit_alvo* **-** Elimina permanentemente todas as alterações dos *commits* desfeitos e quaisquer modificações locais não comitadas.

É recomendado apenas para *commits* locais,
que ainda não foram enviados para um repositório remoto.

Utilize o modo hard (--hard) com extrema cautela,
pois pode resultar em perda irreversível de dados.

- **GIT FETCH:** Verificar as diferenças entre o repositório local e remoto:
  - **git fetch origin :** Baixa alterações do repositório remoto sem mesclar com o repositório local;

- **GIT VIEW FILE HISTORY – VS CODE**: Permite a visualização de todo o histórico de commits de um arquivo específico, para entender como ele evoluiu ao longo do tempo.

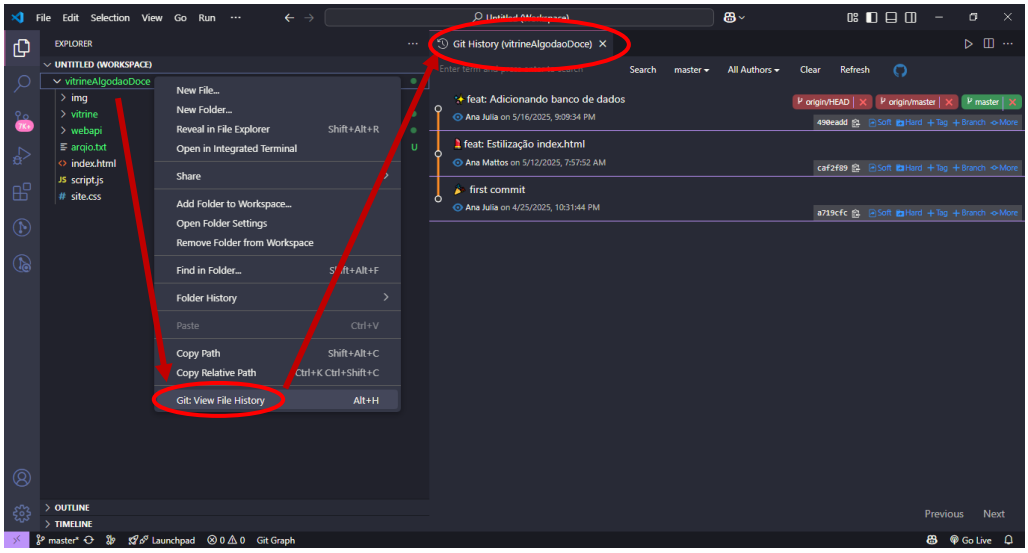

Figura 2 - Git History (projeto)

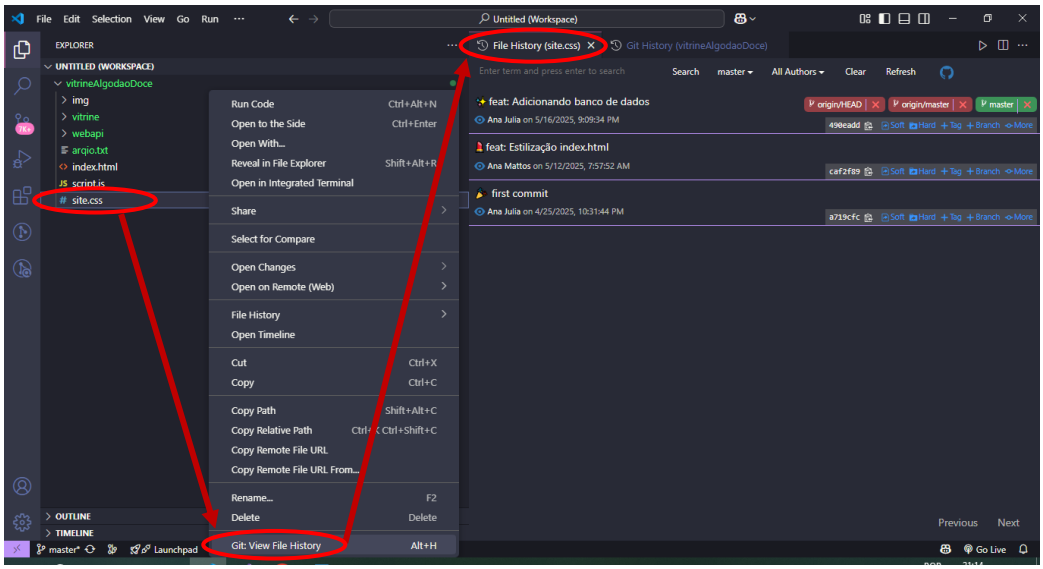

Figura 3 - Git History (arquivo)

Utilidades do Histórico de Arquivos

- **Ver a mensagem do commit**: É possível entender o que foi alterado no arquivo e o motivo da mudança;
- **Ver o autor e a data do commit**: O sistema permite identificar quem realizou a alteração e em que momento ela foi feita;
- **Comparar versões**: É possível comparar a versão atual do arquivo com uma versão anterior (associada a um commit específico). Isso revela as diferenças exatas (*diff*) entre as duas versões;

- **Restaurar uma versão anterior**: É possível reverter o arquivo para um estado correspondente a um commit específico. No entanto, o usuário deve proceder com cautela, pois essa ação pode alterar o histórico do arquivo;
- **Navegar entre commits**: O usuário pode percorrer os diferentes commits do arquivo, acompanhando sua evolução ao longo do tempo.

- **GIT BLAME:** É utilizado para mostrar o que cada linha de um arquivo foi alterada por qual commit e por qual autor
  git blame *nome_do_arquivo*

- **GIT IGNORE**: É um arquivo de texto na raiz do projeto que especifica quais arquivos ou pastas devem ser ignorados**:**

  Como usar o .gitignore?

  - Ao iniciar um diretório, criar um arquivo ***".gitignore"*** no diretório do projeto;
  - Nesse arquivo liste arquivos curingas a serem ignorados, no site ***https://www.toptal.com/developers/gitignore*** é possível criar modelos prontos para serem utilizados

  ✅ **TEST: Adicionando arquivos ao .gitignore**

  Os arquivos adicionados ao .gitignore não são versionados, ou seja, só existe localmente

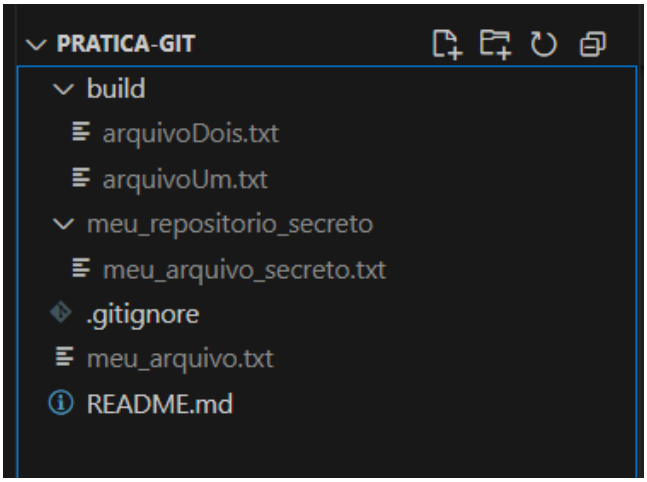

Figura 4 – Repositório Local

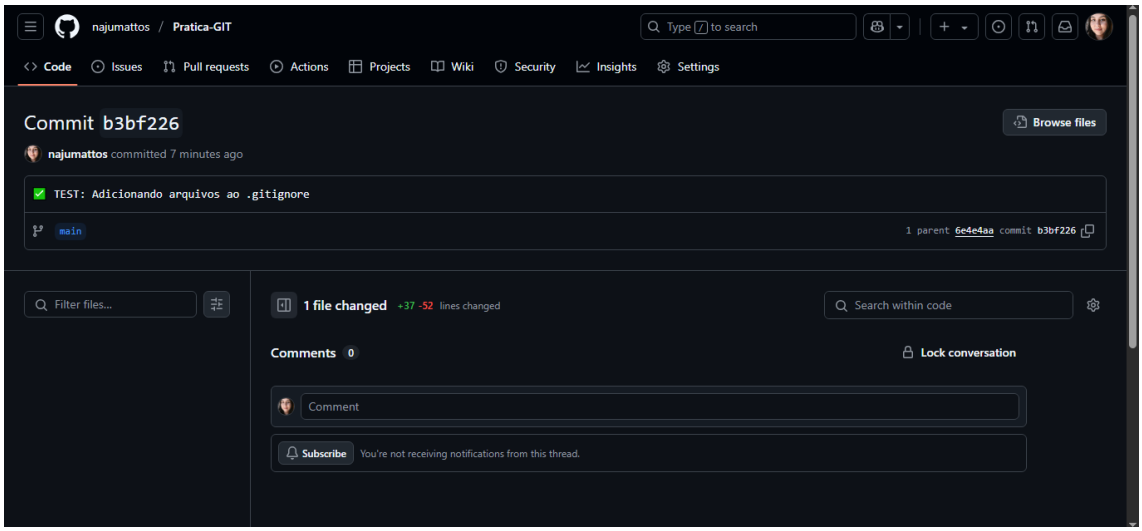

Figura 5 - repositório GitHub

Em quais arquivos utilizar o *.gitignore*?

- Arquivos gerados automaticamente como ***.cache/*** e ***node_modules/;***
- Dados sensíveis ou específicos do ambiente como ***chaves de API*** e ***variáveis de ambiente como Thumbs.db;***
- Arquivos de sistema operacional;
- Arquivos de Configuração de IDEs e ferramentas como ***.vscode/*** e ***.idea/***

Exemplo de arquivo .gitignore

```
# Ignorar arquivos de log
*.log
logs/

# Ignorar diretórios de módulos do Node.js
node_modules/

# Ignorar diretórios de build
build/
dist/

# Ignorar arquivos de cache do Python
__pycache__/
*.pyc

# Ignorar arquivos de sistema operacional
.DS_Store
Thumbs.db

# Ignorar arquivos de configuração da IDE (ex: VS Code)
.vscode/

# Ignorar variáveis de ambiente (se for o caso)
.env
```

Seu uso é recomendado para evitar problemas como:

- Repositórios desnecessariamente grandes;
- Conflitos entre desenvolvedores que usam ambientes ou IDEs diferentes;
- Exposição de informações sensíveis.

1.3. BOAS PRÁTICAS DE COMMITS

- As mensagens devem ser claras e descritivas, escritas no modo Imperativo;
- Usar um padrão de commits;
- Os commits devem ser bem especificados, pequenos e objetivos;
- Commite apenas códigos que funcionem;
- Commite com frequência, mas sem exageros;
- Antes de fazer merge, atualize e revise usando o comando git pull;
- Commite apenas o que for necessário utilizando o comando .gitignore corretamente.

1.4. ESTRUTURA DE COMMIT

:EMOJI: TIPO: MENSAGEM

- **Emoji**
  https://gitmoji.dev/

Escrita do Emoji	Emoji	Tipo	Uso
:tada:	🎉	START	🎉 START: *Projeto inicial com estrutura básica*
:sparkles:	✨	FEAT	✨ FEAT: *Nova Funcionalidade*
:bug:	🐛	FIX	🐛 FIX: *Correção de Bug*
:memo:	📝	DOCS	📝 DOCS: *Alterações na documentação*
:lipstick:	💄	STYLE	💄 STYLE: *Estilo (espaços, identação, ponto e vírgula, etc.)*
:recycle:	♻️	REFACTOR	♻️ REFACTOR: *Refatoração de código, sem alterar comportamento*
:white_check_mark:	✅	TEST	✅ TEST: *Adição ou modificação de testes*
:wrench:	🔧	CHORE	🔧 CHORE: *Tarefas de manutenção (ex: configs, scripts, etc.)*

1.5. BRANCHES

As "Branches" (ramificações) são linhas de desenvolvimento independentes. A comunidade programadora estabeleceu convenções e padrões para o uso de branches, que acabam definindo seus "tipos" pelo propósito que elas servem.

- **Branch Principal (Main/Master):** Deve conter um código estável e funcional, pronto para ser lançado ou que já esteja em produção. Não se desenvolve diretamente nela, exceto em raras situações (como pequenos hotfixes em projetos simples);

- **Branch de Desenvolvimento (Develop):** Contém o histórico de todas as funcionalidades concluídas e integradas que serão incluídas na próxima versão. Por ser a branch de desenvolvimento é menos estável que a main, pois integra trabalhos em andamento;

- **Branches de Funcionalidade (Feature Branches):** O trabalho é isolado nesta branch até que a funcionalidade esteja completa e testada. Uma vez pronta, é mesclada de volta na develop ou main e então excluída. Permitem que vários desenvolvedores trabalhem em paralelo em diferentes funcionalidades sem interferência direta.

  Nomenclatura Comum: **feature/***nome-da-feature*, **feat/***nome-da-feature*, **bugfix/***descrição-do-bug*, **refactor/***descrição;*

- **Branches de Release (Release Branches):** É usada para polimento final, correção de bugs da versão, geração de documentação e outras tarefas de pré-lançamento.

  Nomenclatura Comum: **release/v***(versão)* ou *release-(versão)*;

- **Branches de Hotfix (Hotfix Branches):** É utilizada para corrigir bugs críticos em produção que exigem uma correção imediata e não podem esperar pelo ciclo de desenvolvimento normal da próxima release. São criadas diretamente a partir de main (ou master), pois o bug está na versão em produção.

Nomenclatura Comum: **hotfix/***descrição-do-bug* ou **hotfix/bug-v***(versão)*;

Branches de Suporte (Support Branches): Utilizada para manter linhas de desenvolvimento para versões mais antigas do software que ainda precisam de suporte e correções de bugs, mas não receberão novas funcionalidades.

Nomenclatura Comum: **support/v***(versão)* ou **stable-v***(versão).*

1.6. PROCESSOS GIT

1.6.1. SUBIR PROJETO DO COMPUTADOR PARA O GITHUB

- No site do Github:
    - Criar um novo repositório
    - Copiar LINK-HTTPS
- No GitBash aberto na pasta do projeto:
    - git init
    - git add .
    - git commit -m "MENSAGEM"
    - git remote add origin LINK-HTTPS
    - git pull
    - git push -u origin nome-branch

1.6.2. BAIXAR PROJETO DO GITHUB PARA O COMPUTADOR

- No site do GitHub:
    - Copiar o LINK-HTTPS
- No GitBash aberto na pasta do projeto:
    - Git clone link-https

1.6.3. COMPARANDO REPOSITÓRIOS LOCAL E REMOTO

1.6.4. CONFLITOS

Os conflitos ocorrem quando o Git não consegue integrar automaticamente as alterações de diferentes fontes, exigindo intervenção manual para resolver as diferenças.

Tipos de Conflitos Mais Comuns

GIT MERGE - Ocorrem quando o Git não consegue determinar qual versão do código é a correta:

- **Conflitos de Conteúdo -** O tipo mais comum. Acontece quando duas branches modificaram as mesmas linhas de código no mesmo arquivo de maneiras diferentes, ou quando uma linha foi modificada em uma branch e excluída em outra. O Git marca as seções conflitantes no arquivo, e o usuário precisa editar o arquivo manualmente para decidir qual alteração manter (ou uma combinação das duas);
- **Conflitos de Renomeação/Exclusão –** Ocorre quando um arquivo foi renomeado/excluído em uma branch e modificado/renomeado na outra ou quando duas branches criaram um arquivo com o mesmo nome, mas com conteúdo diferente;
- **Conflitos de Árvore (Tree Conflicts):** Menos comuns, mas podem ocorrer quando há mudanças estruturais no repositório, como a movimentação de diretórios inteiros, e essas mudanças se sobrepõem.

GIT REBASE - Reescreve o histórico de commits, aplicando os commits de uma branch sobre a outra. Isso pode resultar em conflitos semelhantes aos de merge, mas a maneira como eles são resolvidos é um pouco diferente, pois ocorrem commit por commit:

- **Conflitos de Conteúdo:** Assim como no merge, se commits diferentes em ambas as branches modificaram as mesmas linhas no mesmo arquivo, o rebase pausará em cada commit que causa um conflito. O usuário precisará resolver o conflito, adicionar o arquivo resolvido ***git add***, e depois continuar o rebase ***git rebase --continue***. Se o usuário não quiser a alteração do commit, pode pular o commit com ***git rebase –skip***;

- **Conflitos Sequenciais:** Como o rebase aplica os commits um por um, o usuário pode encontrar o mesmo conflito várias vezes se a mesma linha for modificada em vários commits que estão sendo rebaseados.

GIT PULL - É uma combinação de ***git fetch*** (baixa as alterações do repositório remoto) e ***git merge*** (integra essas alterações no branch local). Portanto, os conflitos gerados são essencialmente **conflitos de merge**.

- **Conflitos de Conteúdo:** Se o usuário possui alterações locais não comitadas ou comitadas que entram em conflito com as alterações puxadas do repositório remoto, um conflito de merge ocorrerá. O Git avisará que não pode fazer o merge automático e o usuário terá que resolver o conflito.

GIT STASH - É usado para temporariamente "guardar" as alterações não comitadas, permitindo que o usuário trabalhe em outra coisa e, posteriormente, reaplique essas alterações. Os conflitos podem surgir ao tentar reaplicar um stash:

- **Conflitos ao *git stash apply* ou *git stash pop* -** Se o diretório atual (ou os commits no branch *atual*) tiver sido *modificado* de uma forma que entre em *conflito* com as alterações salvas no *stash*, o Git não conseguirá aplicar o stash automaticamente. Isso ocorre geralmente quando o mesmo arquivo foi modificado na branch após o stash ser criado ou se um arquivo que estava no stash foi excluído/renomeado na branch.

Em todos esses casos, o Git marca as seções conflitantes nos arquivos, e o usuário precisará editar manualmente esses arquivos para resolver os conflitos. Após a resolução, o usuário geralmente usa **git add** *nome_do_arquivo* para marcar o arquivo como resolvido e depois continua a operação.

Como sobreviver a um Conflito?

Certas coisas não da pra se aprender com um passo a passo e a utilidade de se ter este tipo de coisa escrita é zero

As pessoas sabem que iniciantes erram e ninguém fica surpreso quando isso acontece.

1.6.5. README.MD

Readme.md é um arquivo que contém a apresentação e todas as informações do projeto.

O que incluir no readme?

- Título e uma breve descrição sobre o que o projeto faz;
- Tecnologias utilizadas;
- Instruções de instalação e execução do projeto;
- Descrição das principais funcionalidades e características do projeto;
- Licenças e créditos.

REFERÊNCIAS

- ***https://gitmoji.dev/***
- ***https://www.toptal.com/developers/gitignore***
- ***https://github.com/najumattos***
- **https://gemini.google.com** - Este documento foi revisado por IA